KADRAN Yol Haritası: Yeniden Tasarım ve Yeni Özellikler

Oct 22, 2026 · @Yavuz

Özet

Hedeflerin hepsi mevcut kodla yapılabilir; en büyük iş, yeni tasarımı sayfa sayfa değil **önce ortak bir tasarım sistemiyle** kurmak. Bunu yapmazsan 865 satırlık `TaskCreatePage` ve 600 satırlık `TasksPage` gibi dosyaları iki kez yazarsın.

Önerilen sıra: önce temel (Faz 0), sonra bağımsız küçük işler (Faz 1), sonra form ve takvim. Her faz ayrı git dalında, sonunda çalışır halde kalmalı.

Faz	Kapsam	Backend değişikliği	Efor	Önce bitmesi gereken
0	Tasarım sistemi: renk/yazı tokenları, Button, Modal, Toggle, Slider, Skeleton	Yok	Orta	Yok
1	Liste sıralama; kullanıcı adı ve unvan	Var (migration + sorgu)	Küçük	Yok (Faz 0 ile paralel)
2	Görev formu: analog saat, serbest renk, toggle/slider	Küçük (renk doğrulaması)	Büyük	Faz 0
3	Takvim: öncelik kenarlığı, açıklama, popup, boş saat, çizgi işaretçiler	Yok	Büyük	Faz 0
4	Takvimden gün/aralık seçip görev ekleme	Yok	Orta	Faz 2 ve 3
5	İskelet yükleme ve hız	Yok	Orta	Faz 0

Efor göreli bir tahmindir (Küçük: bir akşam, Orta: birkaç akşam, Büyük: bir iki hafta sonu); React'e yeni olduğun için ilk fazlar daha uzun sürer.

Mevcut durum: planı etkileyen gerçekler

Hedeflerinin bir kısmı koda zaten yarım girmiş; bunları sıfırdan değil, geliştirerek yapacağız.

Konu	Kodda ne var	Sonuç
Tasarım altyapısı	Tailwind 3, `tailwind.config.js` boş (`extend: {}`), renkler bileşenlerin içine `slate-900` , `blue-500` diye gömülü	Tema yok; yeniden tasarımın ilk işi token tanımlamak
Bileşen kütüphanesi	Yok. Input stilleri `TaskCreatePage` içinde (`inputClass`), renk seçici orada yerel bileşen	Ortak Button/Modal/Toggle yazılacak
Saat girişi	Doğrudan `<input type="time">` (form içinde 7 yerde)	Analog saat tek bir bileşen olarak yazılıp hepsinde değiştirilebilir
Renk paleti	12 sabit renk + “tip varsayılanı”	Serbest renk için `<input type="color">` veya özel seçici eklenir
Toggle'lar	Düz `<input type="checkbox">` (form, filtre, ayarlar)	Switch bileşeni ile değiştirilir
Slider	Yok	Yeni bileşen (örn. takvim saat aralığı ayarı)
Takvim işaretçileri	OPEN_ENDED ve DEADLINE zaten `marker` olarak tek satır çiziliyor, ama 30 dk'lık kutu gibi **yan yana şerit (lane) hesabına katılıyor**	Çizgiye çevirmek için `layoutDayEvents` 'ten çıkarılmalı
Öncelik gösterimi	Yalnızca küçük ok simgesi (↓ • ↑); sol kenarlık **görev rengini** gösteriyor	Öncelik için ayrı bir görsel kanal gerekli (Faz 3)
Açıklama	`description` verisi var, takvimde gösterilmiyor; modal gösteriyor	Veri ekleme gerekmez
Liste sıralama	Backend sabit sıra kullanıyor (`DEFAULT_ORDER`); özel “serileri tek satırda topla” yolu kendi `sortKey` 'ini kullanıyor; sayfa içinde sadece sabitlenenler öne alınıyor	Sıralama **backend'de** yapılmalı, çünkü liste sayfalı (`hasMore`)
Yükleme durumu	Üç sayfada da yalnızca “Loading…” metni	İskelet yok; veri önbelleği yok

Konu	Kodda ne var	Sonuç
Kullanıcı modeli	`email`, `password`, `timeZone` ve görünüm ayarları; ad/unvan yok	Migration gerekir
Takvimden görev ekleme	Yok; görev yalnızca `/tasks/new` sayfasından oluşturuluyor	Forma ön-doldurma (query parametresi) eklenecek
Testler	Backend'de 41 test (veritabanı gerektirmeden), frontend'de yok	Takvim yerleşim mantığı için test eklemek değerli

Bilmen gereken başka bir kural: `AGENTS.md`, `task.rules.js` (backend) ile `taskRules.js` (frontend) dosyalarının tutarlı kalmasını ve API/davranış değişince `README.md`'nin güncellenmesini istiyor.

Faz 0: Tasarım sistemi temeli

Amac: yeniden tasarımı tek yerden değiştirilebilir yapmak. Renk, köşe yuvarlaklığı veya yazı tipi bir kez değişince tüm uygulama değişmeli.

Ne yapılacak

1. **Karar ver: görünümün karakteri.** Koyu mu açık mı, ya da ikisi de mi? Öneri: renkleri CSS değişkenleriyle tanımla, böylece koyu/açık tema sonradan ucuza eklenir. Bir moodboard (3-4 beyenilen uygulama, örn. Notion Calendar, Linear, Google Takvim) topla.
2. **Token'ları tanımla.** `index.css` içinde `:root { --bg, --surface, --border, --text, --muted, --accent, --danger ... }`; `tailwind.config.js` içinde `theme.extend.colors` bunlara bağlanır (`bg-surface`, `text-muted`). Tip renkleri (COURSE, EXAM...) ve öncelik renkleri de token olur. Mevcut `colors.js` (backend) ile `TYPE_COLORS` (frontend) aynı kalmalı.
3. **Yazı tipi ve ölçek.** Tek bir font (örn. Inter) ve sınırlı boyut listesi (12, 14, 16, 20, 28).
4. **Ortak bileşenler** (`src/components/ui/`): `Button` (primary/ghost/danger), `Modal`, `Switch`, `Slider`, `Field`, `Skeleton`, `Badge`. Yeni sayfalar yalnızca bunları kullanır.
5. **Erişilebilir primitifler** için hazır kütüphane düşün: Radix UI (`@radix-ui/react-dialog`, `react-switch`, `react-slider`, `react-popover`) klavye ve ekran okuyucu desteğini getirir. Kendin yazarsan Esc ile kapatma, odak tuzağı ve `aria` ayrıntıları senin üzerine kalır. Bağımlılık eklemek istemezsen kendi yaz, ama Modal'da bu üç şeyi atlama.
6. **Geçiş stratejisi.** Eski sayfaları bir anda değil, her fazın kendi ekranlarını yeni bileşenlerle yeniden yaz. Bir süre iki stil bir arada olacak; bu normal.

Kontrol: `npm --prefix kadran-frontend run lint` ve `run build` geçiyor; bir "stil sayfası" (`/_ui`, sadece geliştirmede) tüm bileşenleri gösteriyor.

Faz 1: Hızlı kazanımlar

Bu iki iş tasarımdan bağımsız; Faz 0 ile paralel yapılabilir ve backend'de çalışmayı öğrenmek için iyi bir başlangıç.

1A. Görev listesinde sıralama

Sayfalı liste olduğu için sıralama **sunucuda** yapılmalı; taraycıda yapılırsa yalnızca yüklenen 20-30 kayıt sıralanır, ötekiler yanlış yerde kalır. Şu anki `[...tasks].sort(...)` (sabitlenenleri öne alan satır) bu yüzden yanıltıcı: sabitlenmiş bir görev 2. sayfadaysa yukarı çıkmaz.

1. **`task.validator.js` > `listTasks`:** `sort` (`date`, `priority`, `title`, `created`, `type`) ve `order` (`asc`/`desc`) sorgu parametrelerini izin listesiyle doğrula. Bilinmeyen değer 400 döndürsün.
2. **`task.db.js`:** `DEFAULT_ORDER` yerine seçilen sıralamadan `orderBy` üret. Her zaman en sona ikincil anahtar olarak `id` ekle, aksi halde eşit değerler sayfalar arasında yer değiştirir. `priority` bir enum ve PostgreSQL enum'ları tanımlandıkları sırayla (LOW, MEDIUM, HIGH) sıralar; yani desc yönü HIGH'ı öne alır. İlk denemede bunu Prisma Studio ile doğrula.
3. **Seri kısayolu (`collapseSeries`) yolu:** `findCollapsed` kendi `sortKey` fonksiyonuyla sıralıyor; yeni parametreleri burada da uygulamak gerekir, yoksa iki mod farklı davranır. En kolay yol: seçilen sıra için bir karşılaştırma fonksiyonu yazıp ikisinde de kullanmak.
4. **Testler:** `tests/validator.test.js` içine geçersiz `sort` ve `order` değerleri, `task.service.test.js` yanına sıralama testi ekle (sahte Prisma ile çalışıyor).
5. **Frontend (`TasksPage.jsx`):** `filters` durumuna `sort` ve `order` ekle (URL'ye de yazılıyor, `filtersFromSearch` var), bir `<select>` ve yön düğmesi koy; değişince liste sıfırdan yüklenir. Sabitlenenleri öne alma işini ya "sabitlenenler önce" seçeneği olarak sunucuya taşı ya da kaldır.
6. **README'deki** `GET /tasks` parametre tablosunu güncelle.

1B. Kullanıcı adı ve unvan

Adını ve unvanını "title" diye adlandırma; `Task.title` ile karışır. Öneri: `displayName` (görünen ad) ve `headline` (unvan/kısa tanıtım, örn. "Bilgisayar Müh. 3. sınıf"). İkisi de isteğe bağlı.

1. **`schema.prisma` > `User`:** `displayName String?` ve `headline String?` ekle. Alanlar boş bırakılabildiği için mevcut kullanıcılar bozulmaz. Sonra `npx prisma migrate dev --name user_profile`. (Veritabanı Neon ise önce ayrı bir geliştirme dalı/veritabanı kullan.)

2. **`auth.db.js`:** ayarlar `select` listesine iki alanı ekle (şu an `weekStartsOn`, `hideCompleted` gibi tek tek sayılıyor).
3. **`auth.service.js`:** `serializeUser` ve ayar güncelleme fonksiyonuna (`hideCompleted` bölümünün örneğini izle) ekle. Doğrulama: `trim`, örn. ad en fazla 50, unvan en fazla 80 karakter; boş dize `null` olsun.
4. **Kayıt (`registerService`)** isteniyorsa ad alanı opsiyonel olarak al; zorunlu yapma.
5. **Frontend:** `SettingsPage` içine "Profil" bölümü, `Layout` üst menüsünde e-posta yerine ad (yoksa e-posta), dashboard'da "Günaydın, Yavuz" tarzı karşılama. `AuthContext.updateUser` zaten kullanıcıyı güncelliyor.
6. **Testler ve README:** ayar güncelleme testine bir durum ekle, API tablosunu güncelle.

Sonraya bırakabileceğin: profil fotoğrafı (dosya depolama gerektirir; başlangıç harfli renkli daire yeterli).

Faz 2: Görev oluşturma ve düzenleme ekranı

`TaskCreatePage.jsx` dört modu tek dosyada yönetiyor (create, edit-single, edit-occurrence, edit-series) ve 865 satır. UX'i iyileştirmeden önce dosyayı parçalamak gerekir; yoksa her değişiklik diğerini bozar.

Adım 1: Davranışı değiştirmeden parçala. Form durumunu bir `useTaskForm` kancasına (hook), bölümleri (`TypePicker`, `WhenSection`, `RepeatSection`, `DetailsSection`) ayrı bileşenlere taşı. Payload kurma (`handleSubmit`) aynı kalsın. Bitince dört modu elle dene; bu adımın testi budur.

Adım 2: Analog saat (`TimeField`).

- Tek bir bileşen: `<TimeField value="09:30" onChange label />`. Değer her zaman backend'in beklediği `"HH:mm"` metni olsun; böylece validator ve payload hiç değişmez.
- Alana tıklayınca açılan bir popover içinde SVG kadran: önce saat, sonra otomatik olarak dakika seçilir (Material saat seçicisi gibi). İşaretçi için `pointerdown/move/up` olayları ve açı hesabı `Math.atan2(y - merkezY, x - merkezX)`; açıyı saat/dakikaya çevirip 5 dakikaya yuvarla.
- 24 saat düzeni için dairenin içine ikinci halka (13-24) koy; 12 saat seçeneği için AM/PM düğmesi. Kullanıcının `timeFormat` ayarı (`H24` / `H12`) hangisinin gösterileceğini belirlesin.
- **Yazarak girmeyi de bırak.** Kadran hızlı değildir ("14:35" için iki tıklama ve sürükleme). Popover'ın altında saat/dakika metin kutusu olsun; klavye ile oklar (↑↓) çalışsın. Bu, erişilebilirlik için de gerekli.

- Forma 7 yerde geçen `type="time"` alanlarının hepsini bununla değiştir. Başlangıç ve bitiş için tek kadranda iki ibre (başlangıç-bitiş aralığı çizilmiş) ileri bir adım, ilk sürümde gerekmez.
- Yanına **süre önerileri** koy (30 dk, 1 sa, 1,5 sa, 2 sa): bitiş saatini başlangıçtan türetir. Ders ve sınav girerken saat seçmekten çok daha hızlıdır.

Adım 3: İstediğin rengi atayabilme.

- Önemli kalıp: backend bunu şu an **engelliyor**. `task.validator.js` rengi `isValidColor` ile hazır palette karşı kontrol ediyor (hata mesajı: "color must be one of the preset palette values"). `config/colors.js` içinde bunu `^#[0-9a-fA-F]{6}$` biçimine çevir, hem doğrulama hem büyük/küçük harf için küçük harfe normalle. 3 haneli (`#fff`) veya isimli renkleri kabul etme.
- Neden yalnızca 6 haneli hex: `CalendarSpanBar` rengi `${color}40` diye yazıyor (hex'e şeffaflık eklemek için). Yalnızca 6 haneli hex ile çalışır.
- Arabirim: hazır renkler + "Özel" düğmesi (tarayıcının `<input type="color">` seçicisi ya da hex kutusu) + son kullanılan renkler (`localStorage`). Beyaz/çok açık renk seçilirse koyu zeminde görünmez; kontrast uyarısı göster.
- Testler: geçerli hex kabul, geçersiz değer 400 (`validator.test.js`).

Adım 4: Toggle ve slider'lar.

Nerede	Şimdi	Yeni
Tekrarla, sabitle, tamamlanabilir	`checkbox`	`Switch`
Tip, öncelik, zaman türü	düğme / select	Segmented control (3-5 seçenek yan yana)
Takvim saat aralığı (Ayarlar: `dayStartHour` , `dayEndHour`)	iki sayı seçimi	Çift tutamaçlı aralık slider'ı (0-24); backend kuralı "bitiş > başlangıç" slider'da zaten garanti olur
Tekrar süresi	bitiş tarihi seçimi	Tarih + "4 / 8 / 12 hafta" hızlı seçim, üst sınır 1 yıl
Süre (TIMED)	bitiş saati	Süre slider'ı (15 dk adım), bitiş saatini günceller

Slider'ı sadece sürekli değerler için kullan; öncelik gibi 3 seçenekli bir şeyi slider yapmak kötü UX'tir.

Adım 5: Form düzeni. Tipe göre adım adım açılan bölümler (Tip, Ne zaman, Detay, Tekrar), sağda **canlı önizleme kartı**: seçilen renk, öncelik kenarlığı ve saat, takvimde görüneceği gibi gösterilir (Faz 3'teki kart bileşeniyle aynı bileşen). Kaydet düğmesi sabit bir alt

çubukta kalsın. Hata mesajları ilgili alanın altında görünsün (şu an toast).

Kontrol: dört modun hepsinde oluştur/düzenle çalışıyor; tekrarlayan görev oluşturunca doğru sayıda oluşum çıkıyor; yalnızca klavye ile saat girilebiliyor; `npm --prefix kadran-backend test` geçiyor.

Faz 3: Takvimi öne çıkarma

Bu fazda değişen dosyalar: `CalendarWeekView`, `CalendarEventCard`, `CalendarSpanBar`, `CalendarMonthView`, `TaskDetailModal`, `calendarUtils.js`. Backend'e dokunulmaz; gereken veri (`priority`, `description`, `timeType`) zaten `event.originalTask` içinde geliyor.

3.1 Öncelik kenarlığı

Şu an sol kenarlık görev rengini gösteriyor, bu yüzden öncelik **çerçevenin tümü** ile gösterilsin: öncelik üç kademeli ve renge ek olarak kalınlıkla ayrılsın (HIGH 2 px koyu kırmızı, MEDIUM 1 px, LOW kenarlık yok veya ince kesikli). Yalnızca renkle ayırma; renk körü kullanıcılar ve kendi rengini seçen kullanıcılar için kalınlık da bilgi taşımalı. Stil tablosunu bir kez `taskRules.js` içinde (`PRIORITY_STYLE`) tanımla; `CalendarEventCard`, `CalendarSpanBar`, ay görünümü, liste ve dashboard hepsi ondan okusun. Mevcut ↓ • ↑ simgesi kalabilir.

3.2 Açıklamayı göster

- Haftalık blokta kutu yüksekliği yetiyorsa (örn. ≥ 56 px) açıklamanın ilk 1-2 satırını göster (`line-clamp-2`). Kutu küçükse gösterme.
- Üzerine gelince (400 ms gecikmeyle) bir **hover kartı**: tam başlık, saat, açıklama, etiketler, öncelik, hızlı "tamamla" düğmesi. Radix Popover/HoverCard veya kendi `position: fixed` bileşeni.
- **Açık nokta:** bir tekrarlayan görevin oluşumunda açıklama hangi alanda duruyor? Hem `Task.description` hem `TaskSeries.description` var. Hangisinin dolduğuna bak (`serializeTask` ve `TaskDetailModal`'ın `task.series` kullanımı) ve tek bir gösterim kuralı seç (örn. oluşumunki varsa o, yoksa serinin).

3.3 Popup'ları güzelleştir

`TaskDetailModal` ve `ConfirmDialog` Faz 0'daki ortak `Modal` ile yeniden yazılsın: renkli üst şerit (görev rengi), tip ve öncelik rozetleri, tarih ve saat, açıklama bölümü, etiketler, alt kısımda ana eylem (Tamamla), ikincil (Düzenle), tehlikeli (Sil) ayrı stilde. Seri oluşumunda "Bu oluşum / Tüm seri" ayırımı segmented control olsun. Geçiş animasyonu (ölçek + saydamlık, 150-200 ms), Esc ile kapanma, açılınca odak içeride.

3.4 Boş saatler (açık soru)

"Haftalık görünümde boş saatleri gösterme" iki şekilde okunabilir; hangisini istediğini netleştir:

- **A) Boş saatleri daralt:** hiç görevi olmayan saat satırları ince şerit (örn. 14 px) olsun, dolu saatler genişle. Takvim kısalır, haftanın yoğunluğu bir bakışta görülür. **Önerim bu**, bir ayar düğmesiyle açılıp kapansın.
- **B) Boş zamanı vurgula:** görevler arasındaki boşluklara "2 sa boş" gibi etiket koy.

A'nın uygulanması: `CalendarWeekView` şu an sabit `HOUR_HEIGHT = 60` ile `top = dakika * pxPerMinute` hesaplıyor. Bunu `yForMinute(dakika)` fonksiyonuna çevir: her saatin yüksekliğini (dolu 60, boş 14) bir diziden okuyup parçalı toplayan bir hesap. Tüm konumlar (blok, çizgi, saat etiketleri, "↑ earlier" özetleri) bu tek fonksiyondan geçmeli. Bu fonksiyonu saf JS yazıp `calendarUtils.js`'e koy ve test et. Bir saat "dolu" sayılması için haftanın herhangi bir gününde görevle kesişmesi yeterli (sütunlar aynı satır yüksekliğini paylaşır).

3.5 Open-ended ve deadline'lar çizgi olarak, timed'ın üzerinde

Durum: bunlar bugün küçük kutu (`marker`) ve 30 dakikalık yer kaplayıp `layoutDayEvents` içinde yan yana şeritlere katılıyor; yani timed blokları daraltıyorlar. Hedef:

1. `CalendarWeekView` içinde görevleri iki gruba ayır: bloklar (TIMED) ve çizgiler (OPEN_ENDED, DEADLINE). **Yalnızca bloklar** `layoutDayEvents`'e girsin; timed bloklar tam genişlikte kalsın.
2. Çizgiler ayrı bir katmanda, `z-index` blok katmanının üstünde, sütun genişliğinde 2 px yatay çizgi olarak çizilsin; konum `yForMinute(anchorMinutes)`. Çizginin kendisi `pointer-events: none`, tıklanabilir olan sadece sol uca yapışık etiket (başlık, saat, tamamla kutusu). Böylece alttaki timed blok tıklanabilir kalır.
3. Görünüm farkı: **DEADLINE** düz çizgi + sağ ucta bayrak/"due" işareti; **OPEN_ENDED** başlangıçta nokta, aşağıya doğru solan kesikli çizgi (bitişi yok anlamı). İkisi de görev rengini kullansın.
4. Aynı dakikada birden fazla çizgi olursa etiketleri yatayda kaydır (kenar durumu; önce tek çizgiyle çalıştır, sonra ekle).
5. Tamamlanmış çizgi soluklasın; ay görünümü (`compact`) değişmeden kalsın.

3.6 Küçük ama etkili eklemeler

- "Şimdi" çizgisi: bugün sütununda kırmızı yatay çizgi (`yForMinute(şimdiki dakika)`), her dakika güncellenir.
- Takvimi giriş sayfası yapılabilir (zaten `defaultView` ayarı var); dashboard ikinci sıraya iner. Sol filtre paneli daraltılabilir olsun ki takvim genişle.

Kontrol: hafta görünümünde üç durum elle denenir: (1) aynı saatte iki timed + bir deadline, (2) hiç görevi olmayan hafta, (3) gece yarısına yakın görev. `yForMinute` ve şerit yerleşimi için birim testleri yazılır (Vitest, Vite ile uyumlu; frontend'de şu an test yok).

Faz 4: Takvimden gün seçerek görev ekleme

Backend gerekmez: mevcut `POST /tasks` yeterli. İş, takvimde bir seçim yapıp bunu forma ön-doldurma olarak taşımaktır. Faz 2'deki form parçalaması bitmeden başlama.

Davranış (kısa)

Hareket	Sonuç
Ay görünümünde bir güne tıklama	O gün için hızlı ekleme kartı açılır
Ay görünümünde basılı tutup birkaç günün üzerinden sürükleme	Seçilen günler vurgulanır; bırakınca `MULTI_DAY` görev (başlangıç ve bitiş tarihi) ön-doldurulur
Hafta görünümünde boş bir saat aralığını sürükleme	Tek gün içinde `TIMED` görev (başlangıç-bitiş saati ön-doldurulur)
Hafta görünümünde üst "All Day" şeridinde günleri sürükleme	`MULTI_DAY`

Teknik adımlar

1. **Seçim kancası (hook):** `useDragSelect` yaz. `onPointerDown` başlangıç hücresini kaydeder, `onPointerEnter` (basılıyken) bitişi günceller, `onPointerUp` bitirir. Durum: `{ start, end, active }`. Kullanıcı başlangıçtan geriye doğru sürükleyebilir; kaydederken `min` / `max` ile sırala.
2. **Çatışmaları önle:** görev kartlarına tıklayınca (`stopPropagation`) seçim başlamasın; sürükleme sırasında metin seçilmesin (`user-select: none`); bırakma hücrenin dışında olursa `window` üzerinde `pointerup` dinle. Basit tıklama (yer değiştirmeden) tek gün seçimi sayılsın.
3. **Hafta görünümünde saat seçimi:** y koordinatını dakikaya çevir (Faz 3'teki `yForMinute`'in tersi: `minuteForY`), 15 dakikaya yuvarla. Boş saatler daraltılmışsa (3.4) ters fonksiyon parçalı yükseklikleri de bilmeli; bu yüzden iki fonksiyonu birlikte yaz ve test et.
4. **Seçimi göster:** seçilen hücreler yarı saydam renkli, gün sayısı veya "14:00-15:30" etiketi imleçte çıksın.
5. **Ekleme arabirimi:** seçimi bırakınca seçime yapışık bir **hızlı ekleme popover'ı** aç: başlık, tip, (varsa) saat, "Oluştur" ve "Daha fazla seçenek". Sonuncu `/tasks/new?date=2026-10-24&end=2026-10-26&start=14:00&until=15:30` gibi sorgu parametreleriyle formu açar.

6. **Form tarafı:** `TaskCreatePage` `useSearchParams` ile bu parametreleri okuyup başlangıç durumuna yazsın; geçersiz değerleri (bozuk tarih, bitiş < başlangıç) yok say. Zaman türü seçimden çıkarılır: tek gün + saat aralığı = TIMED, birden çok gün = MULTI_DAY, tek gün saatsiz = kullanıcıya sor.
7. **Kaydedince** takvim aynı haftada kalsın ve yeni görev görünsün (`reloadTasks` zaten var). Hızlı ekleme yolunda sayfa değiştirmeden `POST` at, iyimser olarak listeye ekle.

Not: proje masaüstü odaklı olduğu için önce fare ile yap; dokunmatik "uzun basıp sürükle" ayrı bir iş (sayfa kaydırmayla çakışır, `touch-action` ayarlamak gerekir).

Kontrol: ters yönde sürükleme, hafta sınırını aşan seçim (ay görünümünde), Esc ile iptal, mevcut bir göreve tıklamanın hâlâ detay açması.

Faz 5: İskelet yükleme ve hız

İki ayrı şey var: **hızlı görünmek** (iskelet) ve **gerçekten hızlı olmak** (önbellek, daha az istek). Önce ölç, sonra düzelt.

Adım 1: Ölç. Chrome DevTools > Network ile Takvim, Dashboard ve Görevler sayfalarının ilk açılış ve hafta değiştirme sürelerini yaz. Lighthouse'ta bir kez çalıştır. Bu sayılar olmadan neyin düzeldiğini bilemezsin. Hedefi kendin koy (örn. iskelet 100 ms içinde görünsün, hafta değiştirme 300 ms altında hissedilsin).

Adım 2: İskelet bileşenleri. Şu an Takvim (`CalendarPage` ~295. satır), Dashboard (~179) ve Görevler sayfaları yalnızca "Loading..." yazıyor. Her sayfanın **gerçek düzenine benzeyen** bir iskeleti olsun: takvim için boş hafta ızgarası + birkaç soluk blok, dashboard için panel kutuları, liste için satırlar. Tailwind'in `animate-pulse` sınıfı yeterli; Faz 0'daki `Skeleton` bileşeni kullanılsın. İskeletin boyutları gerçek içerikle aynı olmalı ki içerik gelince sayfa zıplamasın.

Adım 3: Hafta değiştirirken içeriği kaybetme. Takvim şu an "bu verinin anahtarı görünür aralıkla eşleşmiyorsa yükleniyor" mantığıyla çalışıyor. İskeleti yalnızca **ilk yükleme**de göster; haftalar arası geçişte eski haftayı hafif soluk tutup yeni veri gelince değiştir.

Adım 4: Önbellek. El ile yazılmış `data / key / reloadTick` yapısı yerine TanStack Query (React Query) düşün: aynı hafta tekrar açılınca anında gösterir (`staleTime`), eski veriyi tutarken yenisini çeker (`placeholderData`), önceki/sonraki haftayı önceden çekebilir (`prefetchQuery`). Öğrenme maliyeti var; Faz 3-4 bittikten sonra, tek sayfada (Takvim) dene. Kullanmak istemezsen en basit alternatif: haftanın sonucunu bir `Map` içinde sakla.

Adım 5: Sayfa bölme. `App.jsx` içinde sayfaları `React.lazy(() => import(...))` ile yükle, `Suspense fallback` olarak ilgili iskeleti ver. `npm run build` çıktısındaki dosya boyutlarıyla öncesi/sonrasını karşılaştır.

Adım 6: Sunucu tarafını kontrol et.

- Veritabanı Neon ise ücretsiz katmanda boşta kalan veritabanı uyuyabilir ve **ilk istek saniyelerce sürebilir**; bu iskeletle de geçilemeyen bir gecikmedir. `DATABASE_URL` için Neon'un bağlantı havuzu (`-pooler`) adresini kullan, sunucu ile veritabanı aynı bölgede olsun.
- Dashboard birden çok `/tasks` isteği atıyor (`Promise.allSettled`). Ölçümün kötü çıkarsa tek bir `/dashboard` uç noktası düşün; önce ölç.
- Takvim sorgusu `from/to` ile `OR` kullanıyor; yavaşsa `EXPLAIN` ile bak, gerekirse `(userId, date)` birleşik indeksi ekle. İndeks eklemek bir migration'dır.

Kontrol: ölçüm tablosunda önce/sonra sayıları var; ağı "Slow 3G"ye alıp her sayfada iskelet görünüyor ve içerik gelince zıplama yok.

Ek fikirler

Hedeflerine ekleyebileceğim, öğrenci uygulaması amaçlı sekiz fikir. Faydası yüksek ve eforu düşük olanlar üstte.

Fikir	Ne sağlar	Efor	Backend
Klavye kısayolları (N yeni görev, T bugün, ←/→ hafta, W/M görünüm)	Takvimi masaüstünde hızlı kullanımı	Küçük	Yok
Sınav geri sayım kartı ("Algoritma sınavına 3 gün")	Dashboard'da EXAM görevleri için en değerli bilgi	Küçük	Yok (`type=EXAM` zaten var)
Açık/koyu tema	Faz 0'daki token'larla neredeyse bedava gelir	Küçük	Yok (tercih `localStorage` veya ayar)
Hafta özeti şeridi ("Bu hafta: 2 sınav, 5 ödev, 12 saat ders")	Haftanın yoğunluğunu bir bakışta gösterir	Küçük	Yok
Geri al (undo) bildirimi	Yanlış silme/tamamlama korkusunu azalttır. Tamamlama kolay; silme için gecikmeli silme veya `deletedAt` alanı gerekir	Orta	Silme için evet
Dersleri toplu ekleme (haftalık program sihirbazı)	Dönem başında 6-8 dersi tek ekranda girmek; mevcut WEEKLY seri mantığını yeniden kullanır	Orta	Yok veya toplu uç nokta

Fikir	Ne sağlar	Efor	Backend
Sürükle-bırak ile görevi taşıma ve süresini uzatma	Takvimi doğrudan manipüle etme hissi; Faz 4'ün doğal devamı	Büyük	Yok (`PATCH /tasks/:id`)
Takvimi `.ics` olarak dışa aktarma	Telefon takvimine/Google Takvim'e taşıma	Orta	Evet (yeni uç nokta)

Bir de **boş durum (empty state)** işi: yeni kullanıcının boş takvimde ne yapacağını gösteren kısa bir yönlendirme ("ilk dersini ekle"). Küçük ama ürünü başkasına gösterirken fark eder.

Riskler ve açık sorular

En büyük risk teknik değil, kapsam: "tüm tasarım değişecek" ile "sekiz özellik" aynı anda yürütülürse hiçbiri bitmez. Aşağıdaki önlemler bunun için.

Risk	Neden olur	Önlem
Yarım kalan yeniden tasarım	Tüm sayfaları bir anda yeniden yazmak	Faz 0 sonrası her faz tek başına yayına çıkabilecek şekilde bitsin; iki stil bir süre bir arada kalabilir
Form bozulması	`TaskCreatePage` dört modu tek dosyada ve 865 satır	Önce davranış değiştirmeden parçala (Faz 2, adım 1); dört modu elle deneme listesi tut
Takvim yerleşim hataları	Çizgiler, daraltılmış saatler ve sürükleme aynı y koordinat hesabını kullanıyor	Tek `yForMinute` / `minuteForY` çifti + birim testi
Veri sözleşmesi tutarsızlığı	Renk, sıralama ve profil API'yi değiştirir	Backend testi + `README.md` API tablosu; `task.rules.js` ile `taskRules.js` senkron (AGENTS.md kuralı)
Migration kazası	`prisma migrate dev` kayma (drift) bulursa veritabanını sıfırlamayı önerebilir	Geliştirme için ayrı veritabanı; kullanıcı verisi olan veritabanında "reset" onayını asla verme; yeni alanlar `null` olabilir olsun
Erişilebilirlik	Analog saat ve sürükleyerek seçim yalnızca fareyle kullanılır	Her ikisinin klavye alternatifi (saat metin kutusu, "günü seç + Enter") bulunsun

Risk	Neden olur	Önlem
Anlamadan kod kabul etmek	Proje yapay zekâ desteğiyle geliştirilmiş ve öğrenmek istiyorsun	Her adımın sonunda değişikliği kendi cümlelerinle açıklayabiliyor musun diye kontrol et; açıklayamadığın kısmı sorup öğren

Açık sorular

- [ ] **Boş saatler:** daraltma (A) mı, boş zaman etiketi (B) mi? (Faz 3.4)
- [ ] **Arayüz dili:** metinler şu an İngilizce (“Edit Series”, “Loading…”). Yeniden tasarımla birlikte Türkçe mi, İngilizce mi, ikisi de mi?
- [ ] **Tema:** koyu mu, açık mı, ikisi birden mi? (Faz 0)
- [ ] **“Unvan” ne demek:** öğrenci sınıfı/bölümü gibi serbest bir metin mi, yoksa seçilen bir rol mu? (Faz 1B, şimdilik serbest metin varsayıldı)
- [ ] **Analog saat:** 24 saatlik tek kadran mı, 12 saat + AM/PM mi? (Faz 2; ayara göre ikisi de desteklenebilir)
- [ ] **Takvimden ekleme:** hızlı popover mı, doğrudan tam forma gitmek mi? (Faz 4; öneri: ikisi, popover “daha fazla” ile forma geçer)

Çalışma yöntemi ve kontrol listesi

Her faz için aynı döngü: dal aç, küçük parçada ilerle, çalıştığını doğrula, birleştir.

1. `git checkout -b faz-1a-siralama` gibi faz başına bir dal aç.
2. Bir adımı bitir, `npm --prefix kadran-backend test`, `npm --prefix kadran-frontend run lint` ve `run build` çalıştır, kısa bir commit at.
3. Elle dene (bu dokümandaki “Kontrol” satırları).
4. API veya davranış değiştiyse `README.md`'yi güncelle.
5. Dalı `main`'e birleştir; sonraki faza geç.

Başlama sırası önerisi: Faz 1A (sıralama) ilk iş olsun; küçük, backend + frontend'i birlikte öğretir, hata riski düşük. Ardından Faz 0 ve 1B paralel gidebilir.

Kontrol listesi

- [ ] Faz 0: renk/yazı token'ları ve `tailwind.config.js` bağlantısı
- [ ] Faz 0: `Button`, `Modal`, `Switch`, `Slider`, `Skeleton`, `Field`
- [ ] Faz 0: stil sayfası (`/_ui`)
- [ ] Faz 1A: `sort` / `order` doğrulaması ve testleri

- [ ] Faz 1A: `task.db.js` `orderBy` + `collapseSeries` yolu
- [ ] Faz 1A: `TasksPage` sıralama kontrolü, README
- [ ] Faz 1B: `displayName` / `headline` migration ve servis
- [ ] Faz 1B: Ayarlar profil bölümü, üst menüde ad
- [ ] Faz 2: `TaskCreatePage` parçalama (davranış aynı)
- [ ] Faz 2: `TimeField` analog saat + klavye girişi
- [ ] Faz 2: hex renk doğrulaması (backend) + özel renk seçici
- [ ] Faz 2: `Switch` /segmented/slider yerleştirme, canlı önizleme
- [ ] Faz 3: `PRIORITY_STYLE` ve öncelik kenarlığı
- [ ] Faz 3: açıklama gösterimi + hover kartı
- [ ] Faz 3: `TaskDetailModal` ve `ConfirmDialog` yenileme
- [ ] Faz 3: `yForMinute` + boş saat daraltma + testler
- [ ] Faz 3: OPEN_ENDED/DEADLINE çizgileri, şimdi çizgisi
- [ ] Faz 4: `useDragSelect` , seçim vurgusu, hızlı ekleme popover'ı
- [ ] Faz 4: formun query parametrelerinden ön-dolması
- [ ] Faz 5: ölçüm, iskeletler, sayfa bölme, önbellek

İstersen sonraki adım: Faz 1A'yı birlikte yapalım; ben kodu yazmadan sen adımları uygular, takıldığın yeri buraya yapıştırırsın, birlikte inceleriz.